

GUÍA PASO A PASO

Integrar Yotsuba Shader en tu motor

Todo lo que necesitas para dar a quienes usan tu framework shaders escritos en C\# con tipado fuerte y validados

Edición 1.0 · Octubre de 2026 · Yotsuba Shader se publica bajo la Microsoft Public License (MS-PL)

Contenido

Antes de empezar **1**
Qué vas a construir 1
Cómo leerla 1
1 Cómo encaja Yotsuba Shader en un motor **2**
1.1 El pipeline 2
1.2 Qué hace Yotsuba Shader y qué hace tu motor 2
1.3 Los cinco puntos de integración 2
1.4 Tres decisiones que debes tomar ahora 3
2 Prepara la solución **4**
2.1 Los proyectos 4
2.2 El paquete del lenguaje 4
2.3 El paso de compilación 4
2.4 Requisitos 5
3 Tus tipos se convierten en tipos de GPU **6**
3.1 Cómo ve la GPU tus tipos 6
3.2 Escalares: no hay que hacer nada 6
3.3 Vectores 6
3.4 Matrices 7
3.5 Texturas 8
3.6 Samplers 8
3.7 Structs y arreglos 9
3.8 Un tipo que no es tuyo 9
3.9 Comprueba que funciona 9
4 Escribe shaders con tus tipos **10**
4.1 Etapas 10
4.2 Slots de interfaz 10
4.3 Elige una sintaxis para tu motor 10
4.4 Un ejemplo completo 11
5 Recursos y la convención de bindings **13**
5.1 Uniforms, storage buffers, texturas y samplers 13
5.2 Slots de binding 13
5.3 Una convención que escala 13
5.4 Datos de uniforms y memoria 14
6 Compila tus shaders **15**
6.1 El generador de código 15
6.2 Dos objetos en cada shader 15
6.3 Los formatos 16
6.4 Del texto a algo que ejecuta la GPU 16
6.5 La línea de comandos 16
6.6 Traducir en tiempo de ejecución 17
7 El lado de CPU: parámetros de shader tipados **18**
7.1 El problema 18
7.2 Paso 1: oculta cómo la API gráfica asigna un uniform 18
7.3 Paso 2: una clase base para cada shader 18
7.4 Paso 3: un shader encima 19
7.5 Paso 4: lógica en la CPU, una vez por draw 19
7.6 Paso 5: falla cuando el shader se carga 20
7.7 Cuando tu motor ya tiene un tipo base 20
8 Carga shaders y crea pipelines **22**

8.1 Una interfaz para la API gráfica 22
8.2 Elige el formato para el dispositivo 22
8.3 El cargador 22
8.4 Qué hace cada backend con un módulo 24
8.5 Uniform buffers y la disposición `std140` 24
9 Materiales y el bucle de render 26
9.1 Usar un shader en un juego 26
9.2 Mantén visible la frecuencia de actualización 26
9.3 Un objeto de shader por material 26
9.4 Notas de rendimiento 26
9.5 Iteración en caliente 27
10 Prueba sin tarjeta gráfica 28
10.1 El lado de CPU es C# normal 28
10.2 Prueba la reflexión 28
10.3 Prueba la disposición 29
10.4 Valida los binarios en CI 29
11 Adóptalo en un motor existente 30
11.1 Paso 1: marca tus tipos 30
11.2 Paso 2: escribe un shader nuevo 30
11.3 Paso 3: ejecuta ambos a la vez 30
11.4 Paso 4: reemplaza los parámetros por cadena 30
11.5 Paso 5: elimina el camino antiguo 30
11.6 Qué medir 30
12 Publícalo 31
12.1 Empaqueta tu motor 31
12.2 Mantén esto estable 31
12.3 Bibliotecas de shaders 31
12.4 Licencias 31
12.5 Documéntalo para tus usuarios 32
13 Solución de problemas 33
Apéndice A: Lista de verificación 34
Apéndice B: Valores built-in en cada sintaxis 35
Apéndice C: Atributos de recursos en cada sintaxis 37
Apéndice D: Glosario 38
Apéndice E: Preguntas frecuentes 39

Antes de empezar

Esta guía es para quien construye un motor, un framework o una herramienta de contenido en C# y quiere que ejecute shaders escritos con Yotsuba Shader. Sigue un motor imaginario, `MyEngine`, desde una solución vacía hasta un paquete probado y publicado. Cada paso es algo que copias en tu propio proyecto.

Qué vas a construir

Al terminar, los programadores que usen tu motor escribirán esto:

Game.cs

```
var lit = new Lit(program.Parameters);

lit.Material = new MaterialData { Roughness = 0.4f };
lit.Albedo = assets.Load<Texture2D>("brick.png");
```

y cada parte de esa línea estará comprobada: el nombre, el tipo y la existencia del parámetro. Para llegar ahí construirás siete cosas:

N.º	Qué construyes	Capítulo
1	Los tipos de tu motor, marcados para que la GPU los entienda	3
2	Las convenciones de slots y de frecuencia de actualización	5
3	El paso de compilación que genera cada shader en los formatos que necesitas	6
4	Una clase base que convierte los uniforms en propiedades tipadas	7
5	Un cargador que crea pipelines a partir del shader compilado	8
6	Pruebas que se ejecutan sin tarjeta gráfica	10
7	Un paquete que tus usuarios instalan	12

Cómo leerla

- Los capítulos 1 y 2 dan el mapa y la preparación. Léelos primero.
- Los capítulos 3 a 5 tratan del lenguaje de shaders y de los datos. Puedes leerlos en cualquier orden cuando necesites un tema.
- Los capítulos 6 a 9 son la integración en sí. Síguelos en orden.
- Los capítulos 10 a 13 tratan de calidad, adopción en un motor existente, publicación y solución de problemas.

Cada listado de código tiene un nombre de archivo arriba. Los listados de esta guía son archivos reales de la carpeta `code/` junto a este documento, y los de C# se compilan contra el paquete del lenguaje cada vez que se construye la guía, así que puedes copiarlos con confianza. La edición en inglés tiene la misma estructura.

> **NOTA**
>
> Un listado muestra el código de un archivo. Cuando un archivo solo muestra algunos miembros, un comentario en la primera línea indica a qué clase pertenecen.

CAPÍTULO 1

Cómo encaja Yotsuba Shader en un motor

1.1 El pipeline

Yotsuba Shader convierte una clase de C# en binarios de shader. Tu motor nunca analiza texto de shaders y nunca incluye un segundo lenguaje. El compilador lee la clase, comprueba sus interfaces y recursos, y produce SPIR-V. Los demás formatos salen de SPIR-V.



Dos traductores están detrás de los otros formatos: SPIRV-Cross escribe HLSL y Metal Shading Language, y Naga escribe WGSL y GLSL. No los llamas directamente. Pides un formato y recibes su texto.

1.2 Qué hace Yotsuba Shader y qué hace tu motor

Yotsuba Shader	Tu motor
Define los atributos que describen tipos, etapas, slots y recursos	Los aplica a sus propios tipos de vector, matriz y textura
Encuentra los shaders de tu proyecto y los comprueba mientras compilas	Decide la convención de bindings y la frecuencia de actualización de cada grupo de datos
Genera cada binario que pides y una descripción de cada shader	Elige en tiempo de ejecución el formato que corresponde a la API gráfica
Expone los mismos dos objetos en cada shader: su código y su reflexión	Crea pipelines, buffers y descriptores a partir de ellos
Ignora el código de CPU de la clase del shader que marcas con un atributo	Da a cada shader una API tipada en la CPU

La línea entre las dos columnas es el tema de esta guía. La columna izquierda ya está hecha. La derecha es lo que construyes tú.

1.3 Los cinco puntos de integración

Punto	Pregunta que responde	Capítulo
Tipos	¿Cuáles de mis tipos son un vector, una matriz, una textura o un sampler?	3
Lenguaje	¿Cómo describen mis usuarios las etapas, las entradas y las salidas?	4
Recursos	¿Adónde va cada uniform, buffer y textura?	5
Compilación	¿Qué formatos genero, y cuándo?	6
API de CPU	¿Cómo asigna el juego un uniform y carga un shader?	7 y 8

1.4 Tres decisiones que debes tomar ahora

Escribe tus respuestas antes de empezar. Cambiarlas después es posible, pero afecta a todos los shaders que tus usuarios hayan escrito.

1. **¿Qué sintaxis recomiendas?** Las sintaxis HLSL, GLSL, MSL, SPIR-V y WGSL expresan la misma intención. La sintaxis no elige la salida. Elige la que la mayoría de tus usuarios ya conoce y úsalo en todos tus ejemplos. Ver el capítulo 4.
2. **¿Qué formatos publicas?** Solo las APIs gráficas que tu motor soporta. Cada formato que quitas hace la compilación más rápida y el paquete más pequeño. Ver el capítulo 6.
3. **¿Cuál es tu convención de bindings?** Una regla que dice qué `set` contiene la cámara, cuál el material y cuál el objeto. El capítulo 5 propone una.

CAPÍTULO 2

Prepara la solución

2.1 Los proyectos

Divide el motor para que los tipos, la capa gráfica y los shaders puedan cambiar a distinto ritmo.

Estructura de la solución

```
MyEngine/
├── MyEngine.sln
├── src/
│   ├── MyEngine.Core/           tipos de vector, matriz y textura
│   │   ├── Vector2.cs  Vector3.cs  Vector4.cs  Matrix4.cs
│   │   └── Texture2D.cs  SamplerState.cs
│   ├── MyEngine.Graphics/       la capa de la API gráfica
│   │   ├── IGraphicsDevice.cs  IParameterSet.cs
│   │   ├── ShaderProgram.cs  Std140.cs
│   │   └── Vulkan/  Direct3D12/  Metal/  OpenGL/
│   └── MyEngine.Shaders/        clase base y shaders del motor
│       ├── ShaderBase.cs
│       └── Lit.cs  Unlit.cs  Tonemap.cs
├── samples/
│   └── Game/                    un juego que usa el motor
└── tests/
    └── MyEngine.Shaders.Tests/
```

Proyecto	Referencia	Por qué
MyEngine.Core	YotsubaShader.Language	Tus tipos llevan los atributos. El paquete solo contiene metadatos, así que no cambia su comportamiento ni cómo se disponen en memoria.
MyEngine.Graphics	MyEngine.Core, YotsubaShader.Language	Lee los shaders compilados y habla con la API gráfica.
MyEngine.Shaders	Core, Graphics, el generador de código	Contiene los shaders que tu motor incluye. El generador los compila durante la compilación.
Game	MyEngine.Shaders	Usa los shaders mediante su API tipada.

2.2 El paquete del lenguaje

El paquete con los atributos va en el proyecto que define tus tipos.

MyEngine.Core.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <!--  Solo metadatos: no cambia tus tipos ni su disposición en memoria  -->
    <PackageReference Include="YotsubaShader.Language" Version="1.*" />
  </ItemGroup>

</Project>
```

2.3 El paso de compilación

El proyecto de shaders referencia el generador de código y lista en `YotsubaTargets` los formatos que publicas.

MyEngine.Shaders.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="YotsubaShader.SourceGenerator" Version="1.*"
                      PrivateAssets="all" />
    <ProjectReference Include="..\MyEngine.Core\MyEngine.Core.csproj" />
    <ProjectReference Include="..\MyEngine.Graphics\MyEngine.Graphics.csproj" />
  </ItemGroup>

  <PropertyGroup>
    <YotsubaTargets>
      spirv;hlsl;msl;wgsl;glsl;glsles
    </YotsubaTargets>
  </PropertyGroup>

</Project>
```

El generador se ejecuta dentro de `dotnet build`. No necesita ninguna herramienta adicional en tu máquina ni ningún paso en tu CI aparte de la propia compilación. Un shader que no compila hace fallar la compilación como cualquier error de C#. El capítulo 6 explica cada destino.

> **CONSEJO**
>
> Usa `PrivateAssets="all"` en el generador. Así queda fuera del paquete que publicas, y tus usuarios no reciben una herramienta de compilación que nunca pidieron.

2.4 Requisitos

- El SDK de .NET 8, en macOS, Linux o Windows.
- Nada más para compilar shaders. Los SDK gráficos que ya usas en tus backends siguen siendo cosa tuya.

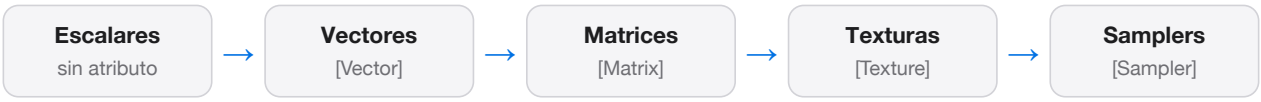
CAPÍTULO 3

Tus tipos se convierten en tipos de GPU

Si construyes un motor, ya tienes `Vector2`, `Matrix4` y `Texture2D`. No necesitas un segundo conjunto de tipos para shaders. Añades atributos a los tipos que ya tienes, y los programadores que usan tu motor escriben shaders con los mismos tipos que usan en todo lo demás.

3.1 Cómo ve la GPU tus tipos

Para la GPU, una struct sin atributos es solo un grupo de campos. El nombre no importa: una struct llamada `Vector2` no es un vector a menos que lo digas. Marcas solo los tipos que deben comportarse como vector, matriz, textura o sampler.



3.2 Escalares: no hay que hacer nada

`bool`, `int`, `uint`, `float` y `double` ya son tipos de GPU. Úsalos solos o dentro de structs.

3.3 Vectores

Pon `[Vector]` en la struct y `[VectorComponent(i)]` en cada componente, empezando en 0. El índice es la posición del componente en el vector de la GPU, así que `X` es 0, `Y` es 1, y así. El número de miembros que marcas es el tamaño del vector.

Vector2.cs

```
using YotsubaShader.Language.Native.Types.Vector;

namespace MyEngine;

[Vector]
public struct Vector2
{
    [VectorComponent(0)] public float X;
    [VectorComponent(1)] public float Y;

    public Vector2(float x, float y) { X = x; Y = y; }

    public static Vector2 operator +(Vector2 a, Vector2 b)
    {
        return new Vector2(a.X + b.X, a.Y + b.Y);
    }
}
```

Tus constructores, operadores y métodos se quedan como están. Los componentes pueden ser campos o propiedades, y un vector puede tener componentes `int` o `uint` cuando todos comparten un tipo.

Vector3.cs

```cs
using YotsubaShader.Language.Native.Types.Vector;

namespace MyEngine;

[Vector]
public partial struct Vector3
{
    [VectorComponent(0)] public float X { get; set; }
    [VectorComponent(1)] public float Y { get; set; }
    [VectorComponent(2)] public float Z { get; set; }

    public Vector3(float x, float y, float z) { X = x; Y = y; Z = z; }

    public static Vector3 Normalize(Vector3 v)
    {
        float length = MathF.Sqrt(v.X * v.X + v.Y * v.Y + v.Z * v.Z);
        return new Vector3(v.X / length, v.Y / length, v.Z / length);
    }
}
```

Vector4.cs

```cs
using YotsubaShader.Language.Native.Types.Vector;

namespace MyEngine;

[Vector]
public struct Vector4
{
    [VectorComponent(0)] public float X;
    [VectorComponent(1)] public float Y;
    [VectorComponent(2)] public float Z;
    [VectorComponent(3)] public float W;

    public Vector4(float x, float y, float z, float w)
    {
        X = x; Y = y; Z = z; W = w;
    }
}
```

IntVectors.cs

```cs
// Vectores enteros: todos los componentes comparten un tipo
[Vector]
public struct Point2
{
    [VectorComponent(0)] public int X;
    [VectorComponent(1)] public int Y;
}

[Vector]
public struct UPoint3
{
    [VectorComponent(0)] public uint X;
    [VectorComponent(1)] public uint Y;
    [VectorComponent(2)] public uint Z;
}
```

CUIDADO

Los componentes de un vector deben tener el mismo tipo, y solo son válidos `float`, `int` y `uint`. Un vector mixto lanza `InconsistentVectorComponentTypeException` y un componente `double` lanza `VectorComponentTypeNotValidException`. Ambos aparecen cuando el compilador procesa un shader que usa el tipo, así que compila un shader que use cada uno de tus tipos antes de publicarlos.

3.4 Matrices

Una matriz es una struct marcada con `[Matrix]` cuyas columnas son vectores. Marca cada columna con `[MatrixColumn(i)]`, empezando en 0. El tipo de la columna es uno de los vectores que declaraste.

Matrix4.cs

```cs
using YotsubaShader.Language.Native.Types.Matrix;

namespace MyEngine;

[Matrix]
public struct Matrix4
{
    [MatrixColumn(0)] public Vector4 Column0;
    [MatrixColumn(1)] public Vector4 Column1;
    [MatrixColumn(2)] public Vector4 Column2;
    [MatrixColumn(3)] public Vector4 Column3;
}
```

3.5 Texturas

`[Texture]` va en una clase o una struct. Necesita dos valores: la dimensión (`D1` , `D2` , `D3` o `Cube`) y el tipo de valores que contiene (`Float` , `Int` o `Uint`). Los ajustes opcionales describen el resto.

Textures.cs

```cs
using YotsubaShader.Language.Native.Types.Texture;

namespace MyEngine;

[Texture(TextureDimension.D2, TextureComponentTypes.Float)]
public class Texture2D { /* la textura de tu motor */ }

[Texture(TextureDimension.Cube, TextureComponentTypes.Float)]
public class TextureCube { /* ... */ }

[Texture(TextureDimension.D2, TextureComponentTypes.Float,
    Depth = TextureDepth.Yes)]
public class DepthTexture2D { /* ... */ }

[Texture(TextureDimension.D2, TextureComponentTypes.Float,
    Usage = TextureUsage.Storage, Format = TextureFormat.Rgba8)]
public class StorageTexture2D { /* ... */ }
```

Ajuste	Valores	Por defecto
`Usage`	`Sampled` (el shader la lee), `Storage` (el shader escribe en ella)	`Sampled`
`Depth`	`No` , `Yes` , `Unknown`	`No`
`Format`	`Unknown` , `Rgba8` , `Rgba16f` , `Rgba32f` , `R32f` , `R32i` , `R32ui` y otros	`Unknown`
`Arrayed`	`true` para un arreglo de texturas	`false`
`Multisampled`	`true` para una textura multimuestra	`false`

Una textura de almacenamiento también necesita su `Format` de píxel, porque el shader escribe en ella.

3.6 Samplers

Un sampler decide cómo se lee una textura. No tiene ajustes: `[Sampler]` solo dice que el tipo lo representa.

SamplerState.cs

```cs
using YotsubaShader.Language.Native.Types;

namespace MyEngine;

[Sampler]
public class SamplerState { /* el sampler de tu motor */ }
```

3.7 Structs y arreglos

Cualquier otra struct se convierte campo por campo, incluidas las structs anidadas. Los arreglos no necesitan atributo. Lo que cambia es de dónde vienen los datos: `[Uniform]` y `[StorageBuffer]` marcan valores que tu motor sube antes de dibujar. El capítulo 5 los cubre.

3.8 Un tipo que no es tuyo

Los atributos van en tipos que controlas. No puedes marcar `System.Numerics.Vector3` ni un tipo de otro paquete. Conserva tu propia struct y convierte en los bordes de tu código. Una struct `partial` te permite añadir las conversiones en otro archivo.

Interop.cs

```
namespace MyEngine;

// No puedes poner atributos a System.Numerics.Vector3: no es tu tipo.
// Conserva tu propia struct y convierte en los bordes de tu código.
public partial struct Vector3
{
    public static implicit operator Vector3(System.Numerics.Vector3 v)
    {
        return new Vector3(v.X, v.Y, v.Z);
    }

    public static implicit operator System.Numerics.Vector3(Vector3 v)
    {
        return new System.Numerics.Vector3(v.X, v.Y, v.Z);
    }
}
```

3.9 Comprueba que funciona

Abre el playground o compila un proyecto pequeño, pon tus tipos en un archivo y un shader que use cada uno en otro. Deben pasar dos cosas: no hay errores y se listan los puntos de entrada del shader. Hazlo una vez por cada tipo que publiques. Es la prueba más barata que escribirás.

CAPÍTULO 4

Escribe shaders con tus tipos

4.1 Etapas

Un shader es una clase. Sus puntos de entrada son métodos marcados con `[ShaderStage]`. Una clase puede tener cualquier número de etapas, y los métodos sin el atributo son C# normal que el compilador ignora.

Etapa	Recibe	Devuelve
`Stage.Vertex`	Entrada por vértice	Una struct que incluye la posición en clip space
`Stage.Fragment`	Los valores interpolados de la etapa vertex	El color para el render target
`Stage.Compute`	Un built-in como el id de invocación global	Nada: escribe en storage buffers

4.2 Slots de interfaz

Cada valor que cruza el límite entre etapas tiene un slot. Hay dos tipos que escribes tú: ubicaciones numeradas y valores built-in.

Types.cs

```
using MyEngine;
using YotsubaShader.Language;
using YotsubaShader.HlslSyntax;

public struct VertexInput
{
    [Location(0)] public Vector3 Position;
    [Location(1)] public Vector3 Normal;
    [Location(2)] public Vector2 UV;
}

public struct VertexOutput
{
    [SV_Position] public Vector4 Position;
    [Location(0)] public Vector3 Normal;
    [Location(1)] public Vector2 UV;
}
```

`[Location(n)]` une una salida de una etapa con la entrada de la siguiente con el mismo número. `[SV_Position]` es un built-in: la posición en clip space que necesita la GPU. Cada sintaxis nombra los built-in a su manera. El apéndice B los lista lado a lado.

4.3 Elige una sintaxis para tu motor

Las cinco sintaxis son equivalentes. Cada una es su propio paquete, con un namespace del mismo nombre, construido sobre `YotsubaShader.Language`. La sintaxis HLSL no exige un destino Direct3D, y la sintaxis WGSL no exige WebGPU. Elige según las costumbres de tus usuarios, no según la API gráfica que soportas.

Si tus usuarios…	Recomienda
vienen de Unity, Direct3D o XNA y MonoGame	HLSL: `YotsubaShader.HlslSyntax`
vienen de OpenGL o Godot	GLSL: `YotsubaShader.GlslSyntax`
escriben para plataformas de Apple	MSL: `YotsubaShader.MslSyntax`
trabajan cerca de Vulkan y SPIR-V	SPIR-V: `YotsubaShader.SpirvSyntax`
escriben para la web	WGSL: `YotsubaShader.WgslSyntax`

Si tus usuarios…	Recomienda
aún no tienen costumbre	HLSL: la relación entre etapas se ve fácil

> **CUIDADO**
>
> Varios nombres existen en más de una sintaxis (`uniform` y `buffer` , por ejemplo). Importa un espacio de nombres de sintaxis por archivo, y pide lo mismo a tus usuarios.

4.4 Un ejemplo completo

El shader de abajo es el que llevarás durante el resto de la guía. Usa los tipos del capítulo anterior, tres grupos de uniforms y una textura con su sampler.

UniformData.cs

```
using MyEngine;

public struct Light
{
    public Vector3 Direction;
    public float Intensity;
}

public struct FrameData
{
    public Matrix4 ViewProjection;
    public Vector3 CameraPosition;
    public float Time;
    public Light Sun;
}

public struct MaterialData
{
    public Vector4 Tint;
    public float Roughness;
}

public struct ObjectData
{
    public Matrix4 World;
}
```

Lit.cs

```
using MyEngine;
using MyEngine.Graphics;
using MyEngine.Shaders;
using YotsubaShader.Language;
using YotsubaShader.Language.Resources;

public partial class Lit : ShaderBase
{
    public Lit(IParameterSet parameters) : base(parameters, Reflection) { }

    // Uniforms: una línea cada uno. El nombre sale de la propiedad.
    [Uniform, Binding(Sets.Frame, 0)]
    public FrameData Frame { get => Get<FrameData>(); set => Set(value); }

    [Uniform, Binding(Sets.Material, 0)]
    public MaterialData Material { get => Get<MaterialData>(); set => Set(value); }

    [Uniform, Binding(Sets.Object, 0)]
    public ObjectData Object { get => Get<ObjectData>(); set => Set(value); }

    // Texturas y samplers
    [Binding(Sets.Material, 1)]
    public Texture2D Albedo { get => GetTexture(); set => SetTexture(value); }

    [Binding(Sets.Material, 2)]
    public SamplerState Sampler { get => GetSampler(); set => SetSampler(value); }

    [ShaderStage(Stage.Vertex)]
    public VertexOutput Vertex(VertexInput input)
    {
        VertexOutput output;
        output.Position = new Vector4(
            input.Position.X, input.Position.Y, input.Position.Z, 1);
```

```
        output.Normal = input.Normal;
        output.UV = input.UV;
        return output;
    }

    [ShaderStage(Stage.Fragment)]
    public Vector4 Fragment(VertexOutput input)
    {
        return Material.Tint;
    }
}
```

Todavía no has visto `ShaderBase`, `Get` y `Set`. Son el lado de CPU del shader y el capítulo 7 los construye. Por ahora, lee los métodos de las etapas y los atributos de las propiedades.

CAPÍTULO 5

Recursos y la convención de bindings

5.1 Uniforms, storage buffers, texturas y samplers

Atributo	Para qué sirve	Tamaño típico
`[Uniform]`	Un valor que se mantiene igual durante un draw o dispatch: una cámara, un tiempo, un material	Pequeño
`[StorageBuffer]`	Un arreglo que el shader puede leer y escribir: partículas, resultados de un cálculo	Grande
Un campo de textura	Los píxeles de una imagen, con `[Texture]` en su tipo	Cualquiera
Un campo de sampler	Cómo se lee una textura, con `[Sampler]` en su tipo	—

Un storage buffer es de lectura y escritura por defecto. Indica el acceso cuando el shader solo lee o solo escribe: `Access = ResourceAccess.ReadOnly` o `WriteOnly`. Un tipo también puede decirlo por ti. `ImmutableArray<T>`, `ReadOnlyMemory<T>` y `ReadOnlySpan<T>` son de solo lectura por lo que son. En un campo de clase usa `ImmutableArray<T>` o `ReadOnlyMemory<T>`, porque C# no permite spans como campos.

> **CUIDADO**
>
> Un campo no puede ser a la vez `[Uniform]` y `[StorageBuffer]`. El compilador lanza `InvalidOperationException`. Elige un uniform para un valor pequeño que no cambia durante el draw, y un storage buffer para un arreglo.

5.2 Slots de binding

Tu motor necesita saber adónde va cada recurso. El atributo compartido `[Binding(set, binding)]` funciona en todas las sintaxis, y cada sintaxis también tiene la grafía de su lenguaje de origen.

El mismo slot en cada sintaxis

```
[Uniform, Binding(1, 0)]                 // cualquier sintaxis
[cbuffer, register(RegisterPrefix.b, 0)]   // HLSL
[uniform, layout(1, 0)]                  // GLSL
[constant, buffer(0)]                    // MSL
[Uniform, DescriptorSet(1), @Binding(0)] // SPIR-V
[uniform, @group(1), @binding(0)]        // WGSL
```

Los slots son solo números. El significado de cada número es tu convención.

5.3 Una convención que escala

Una buena convención hace visible la frecuencia de actualización. Pon lo que cambia menos en un set más bajo. Así el motor enlaza cada set una vez por grupo de draws, no una vez por draw.

Sets.cs

```
namespace MyEngine.Shaders;

// Una regla para todo el motor: lo que cambia menos va en un set más bajo.
public static class Sets
{
    public const int Frame = 0;      // una vez por fotograma: cámara, tiempo, luces
    public const int Material = 1;   // una vez por material: texturas, factores
    public const int Object = 2;     // una vez por objeto: matriz de mundo
}
```

Set	Contiene	Cambia	Se enlaza
0 · Frame	Cámara, tiempo, luces	Una vez por fotograma	Una vez por fotograma
1 · Material	Texturas, samplers, factores del material	Una vez por material	Cuando cambia el material
2 · Object	Matriz de mundo, datos por objeto	Una vez por objeto	Una vez por draw

Pide a tus usuarios que usen estas constantes y nunca números. Un shader escrito con `Sets.Material` sigue funcionando si más adelante reordenas los sets.

5.4 Datos de uniforms y memoria

Una struct de uniforms se sube como memoria simple. Las reglas de cómo se colocan los campos en esa memoria vienen de la API gráfica, no de Yotsuba Shader. Los atributos no cambian la disposición de tus tipos. El capítulo 8 muestra un pequeño ayudante que calcula la disposición estándar `std140` a partir de los mismos atributos que añadiste en el capítulo 3, para que tu cargador de datos y el shader coincidan.

CAPÍTULO 6

Compila tus shaders

6.1 El generador de código

Esta es la forma principal de compilar. Un generador de código de Roslyn se ejecuta como parte de `dotnet build`. Encuentra cada método `[ShaderStage]`, lo compila y escribe el resultado en la otra mitad de la clase. Para esto la clase debe ser `partial`.

Triangle.cs

```
public partial class Triangle
{
    [ShaderStage(Stage.Vertex)]
    public VertexOutput Vertex([SV_VertexID] uint id) { /* ... */ }

    [ShaderStage(Stage.Fragment)]
    public Vector4 Fragment(VertexOutput input) { /* ... */ }
}
```

El generador añade esta mitad. Declara dos campos y los rellena con código generado.

Triangle.g.cs

```
// <generado automáticamente por Yotsuba Shader />
using YotsubaShader.Language.Compilation;

public partial class Triangle
{
    [ShaderIgnore]
    public static readonly ShaderCompiledCode CompiledCode =
        /* ... Código autogenerado con todos los binarios de shader
           que especificaste */;

    [ShaderIgnore]
    public static readonly ShaderReflection Reflection =
        /* ... Código autogenerado con los puntos de entrada y
           los recursos de este shader */;
}
```

Ambos campos llevan `[ShaderIgnore]`, que le dice a Yotsuba Shader que no los trate como parte del shader. La mitad generada no añade ninguna clase base, así que tu propia clase base, como el `ShaderBase` del capítulo 7, nunca entra en conflicto con ella.

6.2 Dos objetos en cada shader

Cada shader expone los mismos dos tipos. Eso permite a tu motor cargar cualquier shader de la misma forma.

YotsubaShader.Language.Compilation

```
public sealed class ShaderCompiledCode
{
    public bool Has(string entryPoint, Binaries format);
    public ReadOnlySpan<byte> Get(string entryPoint, Binaries format);
    public string GetText(string entryPoint, Binaries format);
}

public sealed class ShaderReflection
{
    public IReadOnlyList<Binaries> BinaryFormats { get; }
    public IReadOnlyList<EntryPointInfo> EntryPoints { get; }
    public IReadOnlyList<ResourceBinding> Resources { get; }
}
```

- `ShaderCompiledCode` contiene un binario por punto de entrada y formato. La clave es el nombre del método de C#, así que `nameof(Triangle.Vertex)` lo comprueba el compilador. Los formatos de texto vuelven como bytes UTF-8, y `GetText` los decodifica.

- `ShaderReflection` lista los formatos generados, los puntos de entrada (nombre y etapa) y los recursos (nombre, tipo, acceso y slot). Nunca lees un binario para construir un pipeline.

ReadBinaries.cs

```
using YotsubaShader.Language.Compiler;

// Un punto de entrada, un formato
var vertex = Triangle.CompiledCode.Get(
    nameof(Triangle.Vertex), Binaries.SpirV);

// Cualquier shader, de la misma forma
foreach (var entry in Triangle.Reflection.EntryPoints)
{
    var bytes = Triangle.CompiledCode.Get(entry.Name, Binaries.SpirV);

    // ... la implementación de tus pipelines de gráficos y cómputo
}
```

Los datos viven como datos estáticos en tu ensamblado, así que leerlos no copia ni reserva memoria.

6.3 Los formatos

Destino	`Binaries`	Qué contiene	Se usa para
`spirv`	`Binaries.SpirV`	Binario SPIR-V	Vulkan
`hlsl`	`Binaries.Hlsl`	Texto UTF-8	Direct3D 11 y 12
`msl`	`Binaries.Msl`	Texto UTF-8	Metal
`wgsl`	`Binaries.Wgsl`	Texto UTF-8	WebGPU
`glsl`	`Binaries.Glsl`	Texto UTF-8	OpenGL
`glsles`	`Binaries.GlslEs`	Texto UTF-8	OpenGL ES, WebGL

Solo se generan los destinos listados en `YotsubaTargets`. SPIR-V es el formato que escribe el compilador. HLSL y MSL se traducen desde él con SPIRV-Cross, y WGSL y GLSL con Naga.

6.4 Del texto a algo que ejecuta la GPU

Los formatos de texto aún necesitan un último paso, y es trabajo de la plataforma:

API	Quién compila el texto	Tú publicas
Vulkan	Nadie: SPIR-V es el binario final	SPIR-V
Metal	El framework Metal, al crear la biblioteca	Texto MSL
WebGPU	El navegador	Texto WGSL
OpenGL, WebGL	El controlador gráfico	Texto GLSL
Direct3D 11 y 12	DXC (Shader Model 6) o FXC (Shader Model 5.x), el que ya uses	Texto HLSL

Yotsuba Shader se detiene en el texto HLSL. No produce DXIL ni DXBC. Si quieres bytecode de Direct3D dentro de tu paquete, compila el HLSL con tu propio DXC o FXC en el mismo paso de compilación y publica el resultado.

6.5 La línea de comandos

Para CI, scripts y herramientas de contenido, el mismo compilador está disponible como comando. Le das el archivo de tu proyecto y escribe un binario por punto de entrada.

Terminal

```
yotsuba compile YourGame.csproj \
  --target spirv \
  --output artifacts/shaders
```

La carpeta de salida contiene `Triangle.Vertex.spv`, `Triangle.Fragment.spv` y así. Añade `--entry Triangle.Vertex` para compilar un solo punto de entrada. También es la forma más fácil de inspeccionar la salida con `spirv-val` y `spirv-dis`.

Flujo	Se ejecuta	Ideal para	Salida
Generador de código	Durante `dotnet build`	Motores y juegos que publican shaders con el código	Datos estáticos en una clase `partial`
Línea de comandos	En scripts y CI	Pipelines de contenido, inspección, validación	Archivos `.spv`

6.6 Traducir en tiempo de ejecución

La mayoría de los motores nunca necesita esto: el generador ya produjo todos los formatos. Cuando tu herramienta crea shaders mientras se ejecuta, como un editor, entrega el SPIR-V a la API `Transpile` y recibe texto.

Transpile.cs

```
using YotsubaShader.Transpile;

var hlslOptions = new HlslOptions { ShaderModel = 51, EntryPoint = "Vertex" };
var glslOptions = new GlslOptions { Es = true, Version = 300 };

string hlsl = Transpile.FromSpirVToHlsl(spirv, hlslOptions);
string msl  = Transpile.FromSpirVToMsl(spirv);
string wgsl = Transpile.FromSpirVToWgsl(spirv);
string glsl = Transpile.FromSpirVToGlsl(spirv, glslOptions);
```

Ambos traductores son código nativo, así que cada plataforma necesita su propia compilación. En consolas, traduce en tiempo de compilación: ninguno de los dos traductores se verificó contra una cadena de herramientas de consola.

CAPÍTULO 7

El lado de CPU: parámetros de shader tipados

7.1 El problema

Todo motor asigna un uniform por nombre, normalmente con una cadena:

La forma habitual

```
parameters.Set("Exposure", 1.2f);
parameters.Set("LightDirection", direction);
```

Un error tipográfico, un parámetro renombrado en el shader pero no en el juego, o un valor del tipo equivocado compilan sin errores. Fallan cuando esa línea se ejecuta, no cuando compilas. Yotsuba Shader elimina el problema porque la clase del shader es C#: el nombre y el tipo de cada uniform se escriben una vez, y el compilador de C# los comprueba al compilar el shader y cuando el juego lo usa.

7.2 Paso 1: oculta cómo la API gráfica asigna un uniform

Vulkan, Direct3D, Metal y OpenGL asignan uniforms de formas distintas. Pon esa diferencia detrás de una interfaz. Es la única parte de este capítulo que depende de la API gráfica.

IParameterSet.cs

```
namespace MyEngine.Graphics;

public interface IParameterSet
{
    bool Has(string name);

    // Datos simples: números, vectores, matrices y structs de ellos
    T Get<T>(string name) where T : unmanaged;
    void Set<T>(string name, in T value) where T : unmanaged;

    // Recursos: son objetos de tu motor, no datos simples
    Texture2D? GetTexture(string name);
    void SetTexture(string name, Texture2D texture);

    SamplerState? GetSampler(string name);
    void SetSampler(string name, SamplerState sampler);
}
```

Cada backend la implementa una vez. Una versión de Vulkan escribe en un uniform buffer, y una de OpenGL llama a las funciones de uniforms. Los datos simples pasan por `Get` y `Set`, por eso `T` es `unmanaged`. Las texturas y los samplers son objetos de tu motor y tienen sus propios métodos.

7.3 Paso 2: una clase base para cada shader

La clase base guarda el conjunto de parámetros y da a cada shader los ayudantes que necesita. El nombre del uniform no es un parámetro que escriba el programador: `[CallerMemberName]` hace que el compilador de C# lo rellene con el nombre de la propiedad que llama al ayudante.

ShaderBase.cs

```
using System.Runtime.CompilerServices;
using MyEngine.Graphics;
using YotsubaShader.Language.Compiler;
using YotsubaShader.Language.Compilation;

namespace MyEngine.Shaders;

[ShaderIgnore]
```

```
public abstract class ShaderBase
{
    [ShaderIgnore]
    private readonly IParameterSet parameters;

    [ShaderIgnore]
    protected ShaderBase(IParameterSet parameters, ShaderReflection reflection)
    {
        this.parameters = parameters;

        // Falla al cargar el shader, no en el primer fotograma que dibuja con él
        foreach (var resource in reflection.Resources)
        {
            if (!parameters.Has(resource.Name))
            {
                throw new InvalidOperationException(
                    $"El shader no tiene ningún parámetro llamado '{resource.Name}'.");
            }
        }
    }

    [ShaderIgnore]
    protected T Get<T>([CallerMemberName] string name = "") where T : unmanaged
        => parameters.Get<T>(name);

    [ShaderIgnore]
    protected void Set<T>(T value, [CallerMemberName] string name = "") where T : unmanaged
        => parameters.Set(name, value);

    [ShaderIgnore]
    protected Texture2D GetTexture([CallerMemberName] string name = "")
        => parameters.GetTexture(name)!;

    [ShaderIgnore]
    protected void SetTexture(Texture2D texture, [CallerMemberName] string name = "")
        => parameters.SetTexture(name, texture);

    [ShaderIgnore]
    protected SamplerState GetSampler([CallerMemberName] string name = "")
        => parameters.GetSampler(name)!;

    [ShaderIgnore]
    protected void SetSampler(SamplerState sampler, [CallerMemberName] string name = "")
        => parameters.SetSampler(name, sampler);
}
```

Dentro de `public MaterialData Material { get => Get<MaterialData>(); }` el nombre es `"Material"`. No hay cadena ni `nameof` que olvidar, y renombrar la propiedad renombra el uniform. Todo lo de esta clase es código de CPU, así que lleva `[ShaderIgnore]`.

7.4 Paso 3: un shader encima

El shader del capítulo 4 ya tiene esta forma. Mira sus propiedades otra vez:

Lit.cs (fragmento)

```
[Uniform, Binding(Sets.Material, 0)]
public MaterialData Material { get => Get<MaterialData>(); set => Set(value); }

[Binding(Sets.Material, 1)]
public Texture2D Albedo { get => GetTexture(); set => SetTexture(value); }
```

Una línea por recurso. El constructor pasa la reflexión del shader a la clase base: `base(parameters, Reflection)`. `Reflection` es el campo estático que declara la mitad generada.

7.5 Paso 4: lógica en la CPU, una vez por draw

Un shader ejecuta su código por cada vértice y cada píxel. Todo lo que es igual para todo el draw es más barato en la CPU. Añádelo a la misma clase y márcalo `[ShaderIgnore]`.

LitLogic.cs

```
// Dentro de la clase Lit

// Solo CPU: el compilador la ignora
[ShaderIgnore]
public float Brightness
{
    get => Material.Tint.W;
    set
    {
        var material = Material;
        material.Tint.W = value;
        Material = material;
    }
}

[ShaderIgnore]
public void Fade(float factor) => Brightness *= factor;
```

El juego nunca tiene que saber cómo se guarda el tinte.

7.6 Paso 5: falla cuando el shader se carga

El constructor de `ShaderBase` compara `reflection.Resources` con el conjunto de parámetros. Un recurso que el shader compilado tiene y que el motor no puede proporcionar lanza una excepción al cargar el shader, con el nombre del recurso en el mensaje. Sin esta comprobación, el mismo error aparecería como una pantalla negra en el primer fotograma que dibuja.

7.7 Cuando tu motor ya tiene un tipo base

Algunos motores te dan una clase de la que heredar. La idea no cambia: la propiedad llama al motor, y el resto del juego usa la propiedad.

Motor	Cómo la clase llega a él	Listado
XNA, MonoGame, FNA	Hereda de `Microsoft.Xna.Framework.Graphics.Effect` y usa `Parameters`	LitXna.cs
Unity	Guarda el `Material` en un campo marcado `[ShaderIgnore]`	LitUnity.cs
Godot	Hereda de `ShaderMaterial` y usa `SetShaderParameter`	LitGodot.cs

LitXna.cs

```
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Graphics;
using YotsubaShader.Language.Resources;

public partial class Lit : Effect
{
    // Envuelve el efecto que cargó tu juego
    public Lit(Effect cloneSource) : base(cloneSource) { }

    [Uniform]
    public float Exposure
    {
        get => Parameters[nameof(Exposure)].GetValueSingle();
        set => Parameters[nameof(Exposure)].SetValue(value);
    }

    [Uniform]
    public Vector3 LightDirection
    {
        get => Parameters[nameof(LightDirection)].GetValueVector3();
        set => Parameters[nameof(LightDirection)].SetValue(value);
    }
}
```

LitUnity.cs

```cs
using UnityEngine;
using YotsubaShader.Language.Compiler;
using YotsubaShader.Language.Resources;

public partial class Lit
{
    // Solo CPU: no es parte del shader
    [ShaderIgnore]
    private readonly Material material;

    public Lit(Material material) { this.material = material; }

    [Uniform]
    public float Exposure
    {
        get => material.GetFloat(nameof(Exposure));
        set => material.SetFloat(nameof(Exposure), value);
    }

    [Uniform]
    public Vector3 LightDirection
    {
        get => material.GetVector(nameof(LightDirection));
        set => material.SetVector(nameof(LightDirection), value);
    }
}
```

LitGodot.cs

```cs
using Godot;
using YotsubaShader.Language.Resources;

public partial class Lit : ShaderMaterial
{
    [Uniform]
    public float Exposure
    {
        get => GetShaderParameter(nameof(Exposure)).AsSingle();
        set => SetShaderParameter(nameof(Exposure), value);
    }

    [Uniform]
    public Vector3 LightDirection
    {
        get => GetShaderParameter(nameof(LightDirection)).AsVector3();
        set => SetShaderParameter(nameof(LightDirection), value);
    }
}
```

Un campo que guarda un objeto del motor no es un tipo de GPU, así que lleva `[ShaderIgnore]`. Heredar de un tipo del motor no entra en conflicto con la mitad generada, porque esa mitad no añade ninguna clase base.

CAPÍTULO 8

Carga shaders y crea pipelines

8.1 Una interfaz para la API gráfica

El cargador necesita cuatro cosas de tu capa gráfica. Mantén la interfaz pequeña. Cada backend la implementa una vez.

IGraphicsDevice.cs

```
using YotsubaShader.Language;
using YotsubaShader.Language.Compilation;

namespace MyEngine.Graphics;

public interface IShaderModule : IDisposable { }

public interface IPipelineLayout : IDisposable { }

public interface IGraphicsDevice
{
    GraphicsApi Api { get; }

    IShaderModule CreateShaderModule(
        Stage stage, ReadOnlySpan<byte> code, string entryPoint);

    // Una descripción de cada recurso: la API construye su propio layout
    IPipelineLayout CreatePipelineLayout(
        IReadOnlyList<ResourceBinding> resources);

    IParameterSet CreateParameterSet(IPipelineLayout layout);
}
```

Fíjate en que `CreatePipelineLayout` recibe los recursos directamente de la reflexión. Tu backend no sabe nada de shaders de C#: solo recibe una lista de nombres, tipos, accesos y slots, y construye sus propios descriptor sets, root signature o bind group layout.

8.2 Elige el formato para el dispositivo

Una función asigna a cada API gráfica el formato que tu motor publica para ella.

ShaderFormats.cs

```
using YotsubaShader.Language.Compiler;

namespace MyEngine.Graphics;

public enum GraphicsApi
{
    Vulkan, Direct3D11, Direct3D12, Metal, WebGpu, OpenGl, WebGl,
}

public static class ShaderFormats
{
    public static Binaries For(GraphicsApi api) => api switch
    {
        GraphicsApi.Vulkan => Binaries.SpirV,
        GraphicsApi.Direct3D11 or GraphicsApi.Direct3D12 => Binaries.Hlsl,
        GraphicsApi.Metal => Binaries.Msl,
        GraphicsApi.WebGpu => Binaries.Wgsl,
        GraphicsApi.OpenGl => Binaries.Glsl,
        GraphicsApi.WebGl => Binaries.GlslEs,
        _ => throw new ArgumentOutOfRangeException(nameof(api)),
    };
}
```

8.3 El cargador

`ShaderProgram.Load<TShader>` lee los dos campos estáticos de la mitad generada, elige el formato, crea un módulo por punto de entrada y construye el layout a partir de la reflexión. Guarda el resultado en caché por dispositivo y shader.

ShaderProgram.cs

```
using YotsubaShader.Language.Compilation;

namespace MyEngine.Graphics;

public sealed class ShaderProgram : IDisposable
{
    private static readonly Dictionary<(IGraphicsDevice, Type), ShaderProgram> cache = new();

    private readonly Dictionary<string, IShaderModule> modules;
    private readonly IPipelineLayout layout;

    public IParameterSet Parameters { get; }

    private ShaderProgram(
        Dictionary<string, IShaderModule> modules,
        IPipelineLayout layout,
        IParameterSet parameters)
    {
        this.modules = modules;
        this.layout = layout;
        Parameters = parameters;
    }

    public IShaderModule this[string entryPoint] => modules[entryPoint];

    // ShaderProgram.Load<Lit>(device): la mitad generada de Lit
    // declara dos campos estáticos, CompiledCode y Reflection.
    public static ShaderProgram Load<TShader>(IGraphicsDevice device)
    {
        var key = (device, typeof(TShader));
        if (cache.TryGetValue(key, out var loaded)) return loaded;

        var code = Read<ShaderCompiledCode>("CompiledCode");
        var reflection = Read<ShaderReflection>("Reflection");

        var format = ShaderFormats.For(device.Api);
        if (!reflection.BinaryFormats.Contains(format))
        {
            throw new NotSupportedException(
                $"{typeof(TShader).Name} no se compiló para {device.Api}: " +
                $"agrega '{format}' a YotsubaTargets.");
        }

        var modules = new Dictionary<string, IShaderModule>();
        foreach (var entry in reflection.EntryPoints)
        {
            modules[entry.Name] = device.CreateShaderModule(
                entry.Stage, code.Get(entry.Name, format), entry.Name);
        }

        var layout = device.CreatePipelineLayout(reflection.Resources);
        var program = new ShaderProgram(modules, layout, device.CreateParameterSet(layout));
        cache[key] = program;
        return program;

        static T Read<T>(string field) =>
            (T)typeof(TShader).GetField(field)!.GetValue(null)!;
    }

    public void Dispose()
    {
        foreach (var module in modules.Values) module.Dispose();
        layout.Dispose();
    }
}
```

> **NOTA**
>
> El cargador lee los campos estáticos por nombre. No necesita un registro ni una interfaz en el shader. Si tu motor se compila por adelantado y recorta los miembros no usados, conserva `CompiledCode` y `Reflection` con las anotaciones habituales de recorte.

Si falta el formato, el mensaje le dice a tu usuario qué cambiar: agregarlo a `YotsubaTargets`. Es un ajuste de compilación, no un cambio de código.

8.4 Qué hace cada backend con un módulo

API	Entrada	Crea
Vulkan	Bytes SPIR-V	Un módulo de shader
Direct3D 11 y 12	Texto HLSL, compilado con DXC o FXC	Un objeto de shader o una etapa del pipeline state
Metal	Texto MSL	Una biblioteca y luego una función con el nombre del punto de entrada
WebGPU	Texto WGSL	Un módulo de shader
OpenGL, WebGL	Texto GLSL	Un objeto de shader y luego un programa

8.5 Uniform buffers y la disposición `std140`

El shader lee una struct de uniforms de memoria con una disposición que depende de la API gráfica. La elección común para uniform buffers es `std140`. Sus reglas son pocas: un escalar ocupa 4 bytes. Un vector de dos componentes ocupa 8 bytes y se alinea a 8. Un vector de tres o cuatro componentes se alinea a 16, así que uno de tres componentes ocupa 12 bytes y el siguiente campo puede rellenar el hueco. Una matriz se guarda como columnas de cuatro componentes. Una struct se alinea a 16 y su tamaño se redondea a 16.

No necesitas escribir esos números en ningún sitio. El ayudante de abajo lee los mismos atributos que pusiste en tus tipos en el capítulo 3.

Std140.cs

```
using System.Reflection;
using YotsubaShader.Language.Native.Types.Matrix;
using YotsubaShader.Language.Native.Types.Vector;

namespace MyEngine.Graphics;

// Tamaño y alineación de un tipo en un uniform buffer (reglas std140).
public static class Std140
{
    public static int AlignUp(int value, int alignment)
        => (value + alignment - 1) / alignment * alignment;

    public static (int Size, int Alignment) Measure(Type type)
    {
        if (type == typeof(float) || type == typeof(int) || type == typeof(uint))
            return (4, 4);

        if (type.IsDefined(typeof(VectorAttribute), false))
        {
            int n = Count(type, typeof(VectorComponentAttribute));
            return (n * 4, n == 2 ? 8 : 16);     // un vec3 mide 12 y se alinea a 16
        }

        if (type.IsDefined(typeof(MatrixAttribute), false))
        {
            int columns = Count(type, typeof(MatrixColumnAttribute));
            return (columns * 16, 16);           // cada columna ocupa un vec4 completo
        }

        // Una struct: campos en orden, cada uno con su alineación
        int offset = 0;
        foreach (var field in type.GetFields(BindingFlags.Public | BindingFlags.Instance))
        {
            var (size, alignment) = Measure(field.FieldType);
            offset = AlignUp(offset, alignment) + size;
        }
        return (AlignUp(offset, 16), 16);        // una struct se redondea a 16
    }

    public static int OffsetOf(Type structType, string fieldName)
    {
        int offset = 0;
        foreach (var field in structType.GetFields(BindingFlags.Public | BindingFlags.Instance))
        {
            var (size, alignment) = Measure(field.FieldType);
            offset = AlignUp(offset, alignment);
            if (field.Name == fieldName) return offset;
```

```
            offset += size;
        }
        throw new ArgumentException($"No hay ningún campo llamado '{fieldName}'.");
    }

    private static int Count(Type type, Type attribute)
        => type.GetMembers(BindingFlags.Public | BindingFlags.Instance)
               .Count(m => m.IsDefined(attribute, false));
}
```

Std140Test.cs

```
public struct Particle
{
    public Vector3 Position;
    public float Size;
}

public class Std140Tests
{
    [Fact]
    public void A_vec3_takes_12_bytes_and_the_next_float_fills_the_gap()
    {
        Assert.Equal(0, Std140.OffsetOf(typeof(Particle), nameof(Particle.Position)));
        Assert.Equal(12, Std140.OffsetOf(typeof(Particle), nameof(Particle.Size)));
        Assert.Equal(16, Std140.Measure(typeof(Particle)).Size);
    }

    [Fact]
    public void A_matrix_takes_four_full_columns()
    {
        Assert.Equal(64, Std140.Measure(typeof(Matrix4)).Size);
    }
}
```

CUIDADO

Usa `uint` o `int` para banderas en structs de uniforms, no `bool`. Un `bool` ocupa un byte en la CPU y cuatro en la GPU.

CAPÍTULO 9

Materiales y el bucle de render

9.1 Usar un shader en un juego

Este es todo el código de carga y actualización que necesita un juego. No tiene cadenas ni conversiones.

Game.cs

```
var device = window.CreateDevice(GraphicsApi.Vulkan);

var program = ShaderProgram.Load<Lit>(device);
var lit = new Lit(program.Parameters);

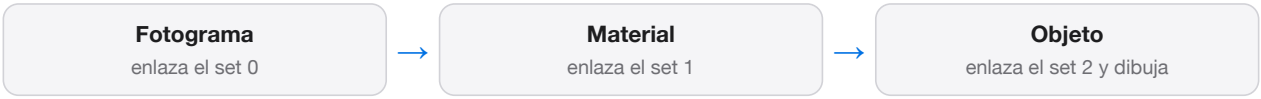
lit.Frame = new FrameData { ViewProjection = camera.ViewProjection, Time = time };
lit.Material = new MaterialData { Tint = new Vector4(1, 1, 1, 1), Roughness = 0.4f };
lit.Albedo = assets.Load<Texture2D>("brick.png");

lit.Material = "rough";   // error CS0029: No se puede convertir implícitamente el tipo 'string' en 'MaterialData'
```

La última línea muestra el sentido del trabajo: un tipo equivocado es un error de compilación en el juego, no un fallo durante un fotograma.

9.2 Mantén visible la frecuencia de actualización

La convención del capítulo 5 se convierte en el bucle de render:



1. Una vez por fotograma, asigna `lit.Frame` y enlaza el set 0.
2. Cuando cambie el material, asigna sus propiedades y enlaza el set 1. Ordena los draws por material para que ocurra lo menos posible.
3. Por cada objeto, asigna `lit.Object` y dibuja.

Como cada grupo es una struct, cada grupo es una sola subida. No asignes los campos de una struct de uniforms uno a uno: construye la struct y asígnala.

9.3 Un objeto de shader por material

`ShaderProgram` es compartido: guarda los módulos y el layout. Un objeto `Lit` guarda los valores de un material. Crea un `Lit` por material y un `ShaderProgram` por shader y dispositivo. Libera los programas cuando el dispositivo desaparezca.

9.4 Notas de rendimiento

- **Resuelve los nombres una vez.** La cadena de `[CallerMemberName]` es una constante que el compilador de C# escribe en tu código, así que el nombre no cuesta nada en ejecución. Haz que tu `IParameterSet` resuelva cada nombre a un desplazamiento o un slot al crearse, a partir de la reflexión, para que cada `Set` posterior sea una escritura en un arreglo y no una búsqueda por texto.
- **Sin boxing.** `Get` y `Set` son genéricos y `unmanaged`, así que un `float` o una struct nunca se encajan en un objeto.
- **Una subida por grupo.** Una struct de uniforms es una sola escritura. Asigna la struct completa y deja que el backend la suba una vez por set.
- **Crea los objetos de material al cargar, no al dibujar.** Un `Lit` es barato, pero un bucle que crea uno por objeto y por fotograma genera basura sin necesidad.

9.5 Iteración en caliente

Durante el desarrollo, la compilación es tu recarga. Cambia el shader, ejecuta `dotnet build` y el generador reescribe los binarios. Si tu editor debe compilar shaders mientras se ejecuta, usa la API de traducción del capítulo 6 sobre el SPIR-V que produce el compilador.

CAPÍTULO 10

Prueba sin tarjeta gráfica

10.1 El lado de CPU es C# normal

Como el shader habla con el motor a través de `IParameterSet`, una prueba puede usar uno falso. La lógica de tus clases de shader se ejecuta en el servidor de compilación, donde no hay tarjeta gráfica.

FakeParameterSet.cs

```
public sealed class FakeParameterSet : IParameterSet
{
    private readonly Dictionary<string, object> values = new();

    public bool Has(string name) => true;

    public T Get<T>(string name) where T : unmanaged
        => values.TryGetValue(name, out var value) ? (T)value : default;

    public void Set<T>(string name, in T value) where T : unmanaged
        => values[name] = value;

    public Texture2D? GetTexture(string name)
        => values.TryGetValue(name, out var value) ? (Texture2D)value : null;

    public void SetTexture(string name, Texture2D texture) => values[name] = texture;

    public SamplerState? GetSampler(string name)
        => values.TryGetValue(name, out var value) ? (SamplerState)value : null;

    public void SetSampler(string name, SamplerState sampler) => values[name] = sampler;
}
```

LitTests.cs

```
public class LitTests
{
    [Fact]
    public void Fade_halves_the_alpha_of_the_tint()
    {
        var lit = new Lit(new FakeParameterSet());
        lit.Brightness = 1;

        lit.Fade(0.5f);

        Assert.Equal(0.5f, lit.Brightness);
    }

    [Fact]
    public void The_material_is_stored_under_its_property_name()
    {
        var parameters = new FakeParameterSet();
        var lit = new Lit(parameters);

        lit.Material = new MaterialData { Roughness = 0.4f };

        Assert.Equal(0.4f, parameters.Get<MaterialData>("Material").Roughness);
    }
}
```

10.2 Prueba la reflexión

Cada shader expone sus recursos. Una prueba que lista los nombres que esperas detecta un uniform renombrado antes que tus usuarios.

ReflectionTests.cs

```
public class ReflectionTests
{
    [Fact]
    public void Lit_exposes_the_resources_the_engine_expects()
    {
        var names = Lit.Reflection.Resources.Select(r => r.Name).ToHashSet();

        Assert.Contains("Frame", names);
        Assert.Contains("Material", names);
        Assert.Contains("Albedo", names);
    }
}
```

10.3 Prueba la disposición

El ayudante `Std140` es código normal, así que las pruebas del capítulo 8 se ejecutan en cualquier sitio. Añade una prueba para cada struct de uniforms de cada shader que publiques: es la diferencia entre una imagen correcta y un desalineamiento silencioso.

10.4 Valida los binarios en CI

La compilación ya falla cuando un shader no compila. Añade un paso más: valida el SPIR-V con `spirv-val` de SPIRV-Tools.

.github/workflows/build.yml

```
name: build
on: [push, pull_request]

jobs:
  shaders:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: 8.0.x

      # El generador de código compila cada shader durante la compilación
      - run: dotnet build -c Release

      # Las pruebas unitarias no necesitan GPU
      - run: dotnet test -c Release --no-build

      # Valida el SPIR-V que produjo la compilación
      - run: |
          sudo apt-get update && sudo apt-get install -y spirv-tools
          yotsuba compile src/MyEngine.Shaders/MyEngine.Shaders.csproj \
            --target spirv --output artifacts/shaders
          for f in artifacts/shaders/*.spv; do spirv-val "$f"; done
```

CAPÍTULO 11

Adóptalo en un motor existente

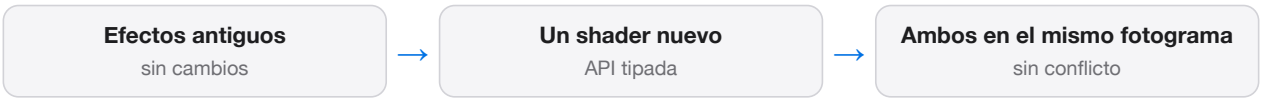
La mayoría de los motores ya tienen efectos, materiales y parámetros por cadena. No tienes que reemplazarlos en un solo paso. Este orden mantiene el motor funcionando después de cada paso.

11.1 Paso 1: marca tus tipos

Añade los atributos del capítulo 3 a los tipos de vector, matriz, textura y sampler que ya tienes. Los atributos son metadatos: nada cambia en ejecución, así que este paso se puede publicar solo.

11.2 Paso 2: escribe un shader nuevo

Elige un shader pequeño y autónomo, como un paso de posprocesado o un material de depuración. Escríbelo con Yotsuba Shader, cárgalo con el cargador del capítulo 8 y dibuja con él. Deja todos los demás efectos como están.



11.3 Paso 3: ejecuta ambos a la vez

El cargador nuevo y el antiguo pueden convivir, porque ambos terminan en la misma API gráfica. Si ya tienes un sistema de materiales, dale un segundo tipo de material que guarde un objeto `ShaderBase` en lugar de un nombre y un diccionario de valores. Dibuja los dos tipos en el mismo bucle de render.

11.4 Paso 4: reemplaza los parámetros por cadena

Busca en el código de tu juego las llamadas que asignan un parámetro por nombre. Cada una es candidata a convertirse en una propiedad:

Antes y después

```
// Antes: una cadena, un valor de cualquier tipo
parameters.Set("Exposure", 1.2f);

// Después: una propiedad, comprobada por el compilador
lit.Exposure = 1.2f;
```

Reemplázalas shader por shader. Después de cada shader, compila: un nombre o un tipo que no coincide ahora es un error del compilador, que es la lista de lo que falta.

11.5 Paso 5: elimina el camino antiguo

Cuando ningún efecto use el formato antiguo, elimina el cargador antiguo y el código de parámetros antiguo. Si los archivos de efectos eran de texto, bórralos: los shaders ahora son archivos C# del proyecto, junto con el resto de tu código.

11.6 Qué medir

- **Tiempo de compilación.** El generador compila shaders durante la compilación. Limita `YotsubaTargets` a los formatos que publicas y compara el tiempo antes y después.
- **Tamaño del paquete.** Cada formato añade sus binarios al ensamblado. Quita los formatos que no uses.
- **Tiempo de fotograma.** No debería cambiar: la GPU recibe el mismo tipo de shader que antes.

CAPÍTULO 12

Publícalo

12.1 Empaqueta tu motor

Tus usuarios referencian un solo paquete. Contiene tus tipos, que llevan los atributos, así que una referencia a `YotsubaShader.Language` viaja con él.

MyEngine.csproj

```
<PropertyGroup>
  <PackageId>MyEngine</PackageId>
  <Version>1.2.0</Version>
  <PackageLicenseExpression>MIT</PackageLicenseExpression>
</PropertyGroup>

<ItemGroup>
  <!--  Tus usuarios reciben los atributos junto con tus tipos  -->
  <PackageReference Include="YotsubaShader.Language" Version="1.*" />
</ItemGroup>
```

Publica los shaders como parte del paquete. Los binarios son datos estáticos dentro de tu ensamblado, así que no hay archivos extra que copiar.

12.2 Mantén esto estable

Tus usuarios escriben shaders contra tus tipos y tus convenciones. Tres cosas los rompen cuando cambian:

- **El orden de los componentes.** El índice 0 es el primer componente. Si `X` deja de ser 0, cada shader que construye un vector cambia de significado.
- **La disposición en memoria que tu motor usa para subir datos.** Los atributos nunca la cambian, así que sigue las reglas de la API gráfica.
- **Los nombres de recursos y los slots.** Renombrar `Frame` o moverlo a otro set rompe cada shader y cada prueba de reflexión.

Trata un cambio en cualquiera de las tres como una versión mayor.

12.3 Bibliotecas de shaders

Una biblioteca de shaders es una biblioteca normal de .NET. Reutiliza tus tipos, construye sobre bibliotecas de otros y comparte las tuyas como paquete. Como los tipos son tuyos, una biblioteca puede describir sus entradas con los tipos que tu motor ya define, y su API de CPU viaja con ella.

12.4 Licencias

Componente	Licencia	Dónde importa
Yotsuba Shader	MS-PL	Tu paquete puede ser comercial o no, sin tarifa
Roslyn	MIT	Dentro del compilador
SPIRV-Cross	Apache-2.0	Si usas `Transpile` para HLSL o MSL
Naga	MIT o Apache-2.0	Si usas `Transpile` para WGSL o GLSL
Gramática SPIR-V de Khronos	MIT	En las tablas del compilador

Conserva los avisos de los componentes de terceros en el archivo `NOTICE` de tu paquete. El nombre y el logotipo de Yotsuba Shader no están cubiertos por la licencia.

12.5 Documéntalo para tus usuarios

Tus usuarios no deberían necesitar esta guía. Dales tres páginas: cómo agregar el paquete, un shader que compila y dibuja, y una lista de tus convenciones (sets, unidades, sistema de coordenadas). Usa tus propios tipos en cada ejemplo.

CAPÍTULO 13

Solución de problemas

Lo que ves	Por qué	Qué hacer
`InconsistentVectorComponentTypeException`	Un vector tiene componentes de tipos distintos	Da a todos los componentes del vector un tipo: `float`, `int` o `uint`
`VectorComponentTypeNotValidException`	Un componente de vector no es `float`, `int` ni `uint`	Usa uno de los tres, por ejemplo `float` en vez de `double`
`InvalidOperationException` en un recurso	Un campo es a la vez `[Uniform]` y `[StorageBuffer]`	Elige uno
Una struct se trata como un simple grupo de campos	El tipo no tiene `[Vector]`, `[Matrix]`, `[Texture]` ni `[Sampler]`	Añade el atributo al tipo que posees, no a un tipo de otro paquete
No se encuentra la clase del shader	Ningún método tiene `[ShaderStage]`, o la clase está en otro proyecto	Marca al menos un método. Solo se inspeccionan los tipos del proyecto que referencia el generador
`CompiledCode` no existe	La clase no es `partial`, o no se referencia el generador	Haz la clase `partial` y referencia el generador
`Binaries.Msl` no está disponible	El formato no está en `YotsubaTargets`	Agrega `msl` y compila de nuevo
`El shader no tiene ningún parámetro llamado …`	El motor no puede proporcionar un recurso que el shader tiene	Agrega el parámetro en el backend o renombra la propiedad
Una pantalla negra sin errores	Una struct de uniforms tiene distinta disposición en la CPU y en la GPU	Calcula los desplazamientos con el ayudante del capítulo 8 y pruébalos
Dos atributos con el mismo nombre	Dos espacios de nombres de sintaxis importados en un archivo	Importa un espacio de nombres de sintaxis por archivo

Apéndice A: Lista de verificación

Úsala antes de publicar una nueva versión de tu motor.

	Elemento
☐	Cada tipo público de vector, matriz, textura y sampler tiene su atributo
☐	Un shader que usa cada uno de esos tipos compila sin errores
☐	`YotsubaTargets` lista cada API gráfica que el motor soporta, y ninguna más
☐	La convención de bindings está documentada y se usa mediante constantes
☐	`ShaderBase` valida la reflexión cuando se carga un shader
☐	Cada struct de uniforms tiene una prueba de disposición
☐	Cada shader tiene una prueba de reflexión con los nombres que el motor espera
☐	CI compila, ejecuta las pruebas y valida el SPIR-V
☐	El paquete lleva los avisos de sus componentes de terceros
☐	Un cambio en el orden de componentes, la disposición en memoria, los nombres de recursos o los slots sube la versión mayor

Apéndice B: Valores built-in en cada sintaxis

El mismo valor tiene un nombre distinto en cada vocabulario. Para el compilador, todos significan lo mismo.

Concepto	HLSL	GLSL	MSL	SPIR-V	WGSL
Etapa vertex					
Posición en clip space (salida del vertex)	`SV_Position`	`gl_Position`	`position`	`BuiltIn_Position`	`@builtin_position`
Índice de vértice	`SV_VertexID`	`gl_VertexIndex`	`vertex_id`	`BuiltIn_VertexIndex`	`@builtin_vertex_index`
Índice de instancia	`SV_InstanceID`	`gl_InstanceIndex`	`instance_id`	`BuiltIn_InstanceIndex`	`@builtin_instance_index`
Vértice base	`SV_StartVertexLocation`	`gl_BaseVertex`	`base_vertex`	`BuiltIn_BaseVertex`	—
Instancia base	`SV_StartInstanceLocation`	`gl_BaseInstance`	`base_instance`	`BuiltIn_BaseInstance`	—
Distancia de recorte	`SV_ClipDistanceN`	`gl_ClipDistance`	`clip_distance`	`BuiltIn_ClipDistance`	`@builtin_clip_distances`
Capa del render target	`SV_RenderTargetArrayIndex`	`gl_Layer`	`render_target_array_index`	`BuiltIn_Layer`	—
Índice de viewport	`SV_ViewportArrayIndex`	`gl_ViewportIndex`	`viewport_array_index`	`BuiltIn_ViewportIndex`	—
Etapa fragment					
Coordenada del fragmento	`SV_Position`	`gl_FragCoord`	`position`	`BuiltIn_FragCoord`	`@builtin_position`
Indicador de cara frontal	`SV_IsFrontFace`	`gl_FrontFacing`	`front_facing`	`BuiltIn_FrontFacing`	`@builtin_front_facing`
Profundidad del fragmento (salida)	`SV_Depth`	`gl_FragDepth`	`depth`	`BuiltIn_FragDepth`	`@builtin_frag_depth`
Profundidad conservadora	`SV_DepthGreaterEqual` / `SV_DepthLessEqual`	—	`depth("greater")` / `depth("less")`	—	—
Referencia de stencil (salida)	`SV_StencilRef`	`gl_FragStencilRefARB`	`stencil`	`BuiltIn_FragStencilRef`	—
Índice de muestra	`SV_SampleIndex`	`gl_SampleID`	`sample_id`	`BuiltIn_SampleId`	`@builtin_sample_index`
Máscara de muestras	`SV_Coverage`	`gl_SampleMask` / `gl_SampleMaskIn`	`sample_mask`	`BuiltIn_SampleMask`	`@builtin_sample_mask`
Índice de primitiva	`SV_PrimitiveID`	`gl_PrimitiveID`	`primitive_id`	`BuiltIn_PrimitiveId`	`@builtin_primitive_index`
Coordenadas baricéntricas	`SV_Barycentrics`	`gl_BaryCoordEXT`	`barycentric_coord`	`BuiltIn_BaryCoord`	—
Coordenadas baricéntricas (sin perspectiva)	—	`gl_BaryCoordNoPerspEXT`	—	`BuiltIn_BaryCoordNoPersp`	—
Invocación auxiliar	—	`gl_HelperInvocation`	—	`BuiltIn_HelperInvocation`	—
Posición de la muestra	—	`gl_SamplePosition`	—	`BuiltIn_SamplePosition`	—
Etapa compute					
Id de invocación global	`SV_DispatchThreadID`	`gl_GlobalInvocationID`	`thread_position_in_grid`	`BuiltIn_GlobalInvocationId`	`@builtin_global_invocation_id`
Id de invocación local	`SV_GroupThreadID`	`gl_LocalInvocationID`	`thread_position_in_threadgroup`	`BuiltIn_LocalInvocationId`	`@builtin_local_invocation_id`

Concepto	HLSL	GLSL	MSL	SPIR-V	WGSL
Índice de invocación local	`SV_GroupIndex`	`gl_LocalInvocationIndex`	`thread_index_in_threadgroup`	`BuiltIn_LocalInvocationIndex`	`@builtin_local_invocation_index`
Id del grupo de trabajo	`SV_GroupID`	`gl_WorkGroupID`	`threadgroup_position_in_grid`	`BuiltIn_WorkgroupId`	`@builtin_workgroup_id`
Número de grupos de trabajo	—	`gl_NumWorkGroups`	`threadgroups_per_grid`	`BuiltIn_NumWorkgroups`	`@builtin_num_workgroups`
Tamaño del grupo (atributo de método)	`numthreads`	`layout_local_size`	—	—	`@workgroup_size`
Tamaño del grupo (valor built-in)	—	`gl_WorkGroupSize`	`threads_per_threadgroup`	`BuiltIn_WorkgroupSize`	—
Tamaño del subgrupo	—	`gl_SubgroupSize`	`threads_per_simdgroup`	`BuiltIn_SubgroupSize`	`@builtin_subgroup_size`
Id de invocación del subgrupo	—	`gl_SubgroupInvocationID`	`thread_index_in_simdgroup`	`BuiltIn_SubgroupInvocationId`	`@builtin_subgroup_invocation_id`
Id del subgrupo	—	`gl_SubgroupID`	`simdgroup_index_in_threadgroup`	`BuiltIn_SubgroupId`	`@builtin_subgroup_id`
Número de subgrupos	—	`gl_NumSubgroups`	`simdgroups_per_threadgroup`	`BuiltIn_NumSubgroups`	`@builtin_num_subgroups`

Apéndice C: Atributos de recursos en cada sintaxis

Concepto	HLSL	GLSL	MSL	SPIR-V	WGSL
Uniform buffer	`cbuffer, ConstantBuffer<T>`	`uniform`	`constant`	`Uniform`	`uniform`
Storage buffer de lectura y escritura	`RWStructuredBuffer<T>`	`buffer`	`device`	`StorageBuffer`	`storage(storage_use.read_write)`
Storage buffer de solo lectura	`StructuredBuffer<T>`	`buffer`	`device(readOnly: true)`	`StorageBuffer`	`storage(storage_use.read)`
Slot de binding	`register(RegisterPrefix.b, n)`	`layout(set, binding)`	`buffer(n), texture(n), sampler(n)`	`DescriptorSet(s) + Binding(n)`	`@group(s) + @binding(n)`
Slot de textura	`register(RegisterPrefix.t, n)`	`layout(set, binding)`	`texture(n)`	`DescriptorSet(s) + Binding(n)`	`@group(s) + @binding(n)`
Slot de sampler	`register(RegisterPrefix.s, n)`	`layout(set, binding)`	`sampler(n)`	`DescriptorSet(s) + Binding(n)`	`@group(s) + @binding(n)`

Apéndice D: Glosario

Término	Significado
Punto de entrada	Un método marcado con `[ShaderStage]`. La GPU empieza a ejecutar una etapa ahí
Etapa	Un paso del pipeline: vertex, fragment o compute
Location	Un número que une una salida de una etapa con una entrada de la siguiente
Built-in	Un valor que el pipeline proporciona o consume, como la posición en clip space
Uniform	Un valor que se mantiene igual durante un draw o dispatch
Storage buffer	Un arreglo que un shader puede leer y escribir
Set y binding	Dos números que dicen dónde se enlaza un recurso
Reflexión	La descripción de un shader: sus formatos, puntos de entrada y recursos
SPIR-V	El formato binario de shaders que escribe el compilador. Los demás formatos se traducen desde él
SPIRV-Cross, Naga	Los traductores que escriben HLSL y MSL, y WGSL y GLSL
`std140`	Un conjunto de reglas sobre cómo se colocan los campos de una struct en un uniform buffer
`[ShaderIgnore]`	Un atributo que le dice a Yotsuba Shader que un miembro es código de CPU, no parte del shader

Apéndice E: Preguntas frecuentes

Pregunta	Respuesta
¿Mi juego necesita Roslyn o el compilador en tiempo de ejecución?	No. El compilador se ejecuta durante la compilación. Al juego solo llegan los binarios generados, que son datos estáticos en tu ensamblado.
¿Puedo publicar solo SPIR-V y traducir en tiempo de ejecución?	Sí, con la API `Transpile`. Está pensada para editores y herramientas. Para un juego publicado, traduce en tiempo de compilación y evita llevar un traductor en el reproductor.
¿Qué formatos necesito para un motor Direct3D?	Solo `hlsl`. Compila el texto con el DXC o FXC que ya uses. Yotsuba Shader no produce DXIL ni DXBC.
¿Pueden dos shaders usar sintaxis distintas?	Sí. La sintaxis no cambia el resultado. Importa un espacio de nombres de sintaxis por archivo.
¿Tengo que hacer mis clases de shader `partial`?	Solo cuando usas el generador de código, porque añade la otra mitad de la clase. La línea de comandos no lo necesita.
¿A qué se aplica `[ShaderIgnore]`?	A cualquier miembro o tipo que sea código de CPU: campos, propiedades, métodos y clases enteras, como una clase base.
¿Pueden convivir mi clase base y la mitad generada?	Sí. La mitad generada no añade ninguna clase base, así que sirve cualquier clase de la que heredes, incluido el tipo de otro motor.
¿Y si mi API gráfica no está en la lista?	Elige el formato más cercano de `YotsubaTargets`. SPIR-V siempre está disponible, y los formatos de texto cubren casi todas las APIs.
¿Puedo leer SPIR-V de vuelta a C#?	Esa dirección está planeada y todavía no tiene API.